

AADK 8: Task 1 — Automated Testing Deep Dive & Mapping

Student Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – Session 8

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

1. Research & Definitions

Unit Test

A **unit test** verifies the correctness of a small piece of logic such as a function or class in isolation without involving the Android UI or device components.

Example Scenario:

Testing a function that calculates the next artwork index in the Art Space app when the **Next button** is pressed.

Instrumented UI Test

An **instrumented UI test** runs on an Android device or emulator and verifies how UI components behave when users interact with them.

Example Scenario:

Testing whether clicking the **Next Artwork button** actually changes the artwork image and title displayed on the screen.

ComposeTestRule

ComposeTestRule is a testing utility in Jetpack Compose that allows developers to **set Compose UI content and interact with UI elements during testing**.

Example Scenario:

Using ComposeTestRule to launch the Art Space screen and simulate clicking the **Next button**.

Semantics Tree

The **semantics tree** represents the accessibility and testing structure of Compose UI elements, allowing tests to locate UI components by labels, text, or tags.

Example Scenario:

Finding the **Next button** using a test tag in the semantics tree and verifying that the click action updates the displayed artwork.

2. Selected UI Feature to Test

Feature

Next Artwork Button (Art Space App)

UI Elements Involved

- **Button** – Next Artwork
- **Image** – Artwork Display
- **Text** – Artwork Title / Artist

Expected Behavior

When the user taps the **Next Artwork** button:

1. The displayed artwork image should change.
 2. The artwork title text should update.
 3. The artwork artist text should update.
 4. The artwork index/state should increment.
-

3. Test Strategy Document

Test Type	What to Test	Setup Needed	Assertion / Verification
Unit Test	Logic that updates artwork index	Create ViewModel or function that increments artwork index	Verify next artwork index is returned correctly
Instrumented UI Test	User clicks Next button	Launch Compose screen using ComposeTestRule	Perform click on button

Test Type	What to Test	Setup Needed	Assertion / Verification
Instrumented UI Test	Artwork image changes	Use test tag for Image component	Verify new image is displayed
Instrumented UI Test	Artwork title text updates	Locate Text node in semantics tree	Assert that new title text appears
Instrumented UI Test	Artist name updates	Locate Text node	Verify artist text changes correctly

Example Test Flow

1. Launch Art Space screen using **ComposeTestRule**
2. Locate **Next button** using semantics tree or test tag
3. Perform **click action**
4. Verify that:
 - Image displayed is different
 - Artwork title text changed
 - Artist name text updated

4. Reflection

Writing tests early improves code quality because it ensures that each feature behaves as expected before the app grows more complex. Tests help detect bugs quickly and make it easier to modify code without breaking existing functionality. If the application state changes but the UI is not tested, users may see incorrect information or broken interactions. One challenge in writing Compose UI tests is correctly identifying UI elements in the semantics tree and managing asynchronous state updates during testing.